

Cryptography Algorithms and Verilog Implementation 130330

Permutation (Assignment 4)

Date: Sunday, April 12, 2026

Submission:	By not more than 2 people together
Deadline:	As instructed by the lecturer – via moodle
According to JCT's policy, if you violate any submission or deadline criteria you disqualify your assignment	

A. About this Exercise

In this assignment we will exercise the Verilog implementation and simulation of the *permutation* taken from the DES algorithm.

This particular *permutation* (and its inverse) is described in the *Data Encryption Standard (DES), FIPS PUB 46-3* (given with this exercise). See page 10 of that document for detailed description and page 9 of that document to where the *permutation* and its inverse are used in the algorithm flow.

B. Your Mission

Your mission in this exercise is to write three Verilog files:

1. Verilog implementation of the *permutation*
2. (Another) Verilog implementation of the *inverse permutation*
3. Testbench that exercises both of the designs together

You will then simulate the testbench with both designs in ModelSim

C. Instructions on How to Implement the Permutation and the Inverse Permutation

Write the Verilog code according to the example given in class.

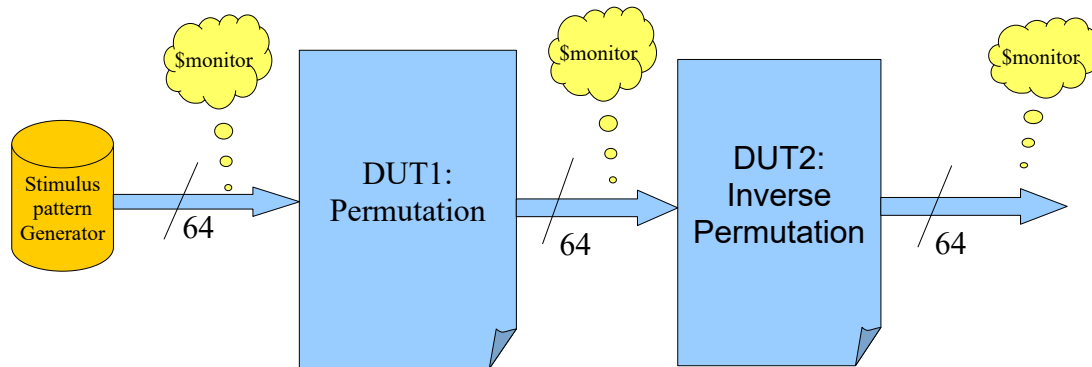
You can take the permutation details directly from page 10 of the *Data Encryption Standard (DES), FIPS PUB 46-3* document. Nevertheless, we recommend that instead of a tedious manual writing, you should take an already written numbers straight from C implementation given at the following links (and prepare the Verilog code by using automatic scripting such as Matlab, Perl, Python or alike):

- <http://www.ubiqx.org/proj/libcifs/source/Auth/DES.c>
- <http://www.codemiles.com/c-code-examples/data-encryption-standard-des-using-c-language-t10770.html>

There are a number of legitimate ordering conventions such as [63:0] or [64:1] and alike... choose one of them, but don't mess up with other conventions.

D. Instructions on How to Write the Testbench

One of the commonly used practices used to confirm the correctness of an encryption algorithm is to check it with its inverse. Your testbench architecture will look similar to the following diagram:



The first pattern that we want you to try is a circular left rotation of the 64'h0123456789abcdef pattern (i.e., 64'h0123456789abcdef, 64'h123456789abcdef0, 64'h23456789abcdef01 etc.) You can do it manually or automatically in the Verilog code.

Discuss with your lecturer in lab about a thorough and efficient test pattern mechanism and implement it in your testbench as well.

E. Your Lab Report Must Include

1. The Verilog code for the permutation, inverse permutation and testbench
2. Output log of the \$monitor tracking during simulation
3. Selected waveforms of the 64'h0123456789abcdef rotating pattern